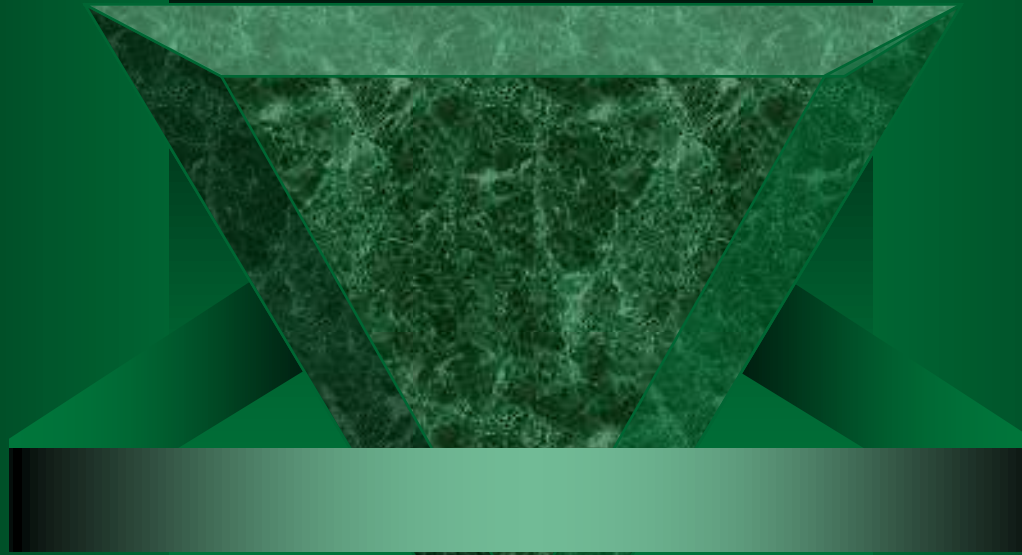


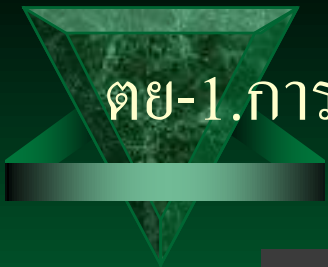
Apirak :



6.2 Recursive subroutine



- t Recursion is the technique of making a function call itself. This technique provides a way to break complicated problems down into simple problems which are easier to solve.



ตย-1.การใช้ recursive แก้ปัญหาแทนการทำ Loop

```
1
2 program ForLoopExample;
3   var i: Integer;
4   begin
5     for i := 1 to 3 do
6     begin
7       WriteLn('Line No.', i);
8     end;
9   end.
10
```



ตย-2.การใช้ recursive แก้ปัญหาแทนการทำ Loop: Factorial Problem

$$4! = 4 \times 3 \times 2 \times 1$$

$$n! = \prod_{k=1}^n k$$

$$n! = n \cdot (n - 1)! \quad (n > 0)$$
$$0! = 1$$

recursive function
Base case



Factorial Problem

factProd.pas

$$4! = 4 \times 3 \times 2 \times 1$$

$$n! = \prod_{k=1}^n k$$

```
1  program FactorialProduct ;
2
3  var
4      .. n, k : Integer ;
5      .. fact : Integer ;
6
7  begin
8      .. Write ('Enter n: ') ;
9      .. ReadLn (n) ;
10
11     .. fact := 1 ; .. {ค่าเริ่มต้น = empty product}
12
13     .. for k := 1 to n do
14         .. fact := fact * k ;
15
16     .. WriteLn (n, '! = ', fact) ;
17 end.
```

no return value function (procedure)

```
1 program <ProgramName>;
2
3 {==== Variable Declarations ====}
4 var
5     <identifier> [, <identifier> ...] : <type>;
6     { repeat as needed }
7
8 {==== Procedure Declaration ====}
9 procedure <ProcedureName>(<param1> : <type>; <param2> : <type>);
10 begin
11     <statement_list>;
12 end;
13
14 {==== Main Program Block ====}
15 begin
16     <statement>;
17     ...
18 end.
```

```
1  program Example;
2
3  var
4      num1:Integer; { global }
5
6  procedure ShowResult(x: Integer);
7  begin
8      writeln('The result is: ', x);
9  end;
10
11 procedure ShowTitle();
12 begin
13     writeln('Title');
14 end;
15
16 begin
17     num1 := 10;
18     ShowTitle();
19     ShowResult(num1);
20     ShowResult(20);
21
22 end.
```

1

2

Execution flows : Function in Pascal(1)

outine)

main.pas

```
1 program <ProgramName>;
2
3 {==== Variable Declarations ====}
4 var
5   <identifier> [, <identifier> ...] : <type>;
6
7 {==== Function Declaration ====}
8 function <FunctionName>(<param1> : <type>; <param2> : <type>) : <return_type>;
9 begin
10   exit ( <expression> );
11 end;
12
13 {==== Main Program Block ====}
14 begin
15   <statement>;
16   ...
17 end.
```

main.pas

```
1 program PowerA;
2 var
3   v1:Integer;
4 {==== Function Declaration ====}
5 function PowerA(a: Integer): Integer;
6 begin
7   { immediately return a squared value }
8   Exit(a * a);
9 end;
10
11 {==== Main Program Block ====}
12 begin
13   { call PowerA(2) → returns 4 → assign to v1 }
14   v1 := PowerA(2);
15 end.
```

1

2

3



Implementing a Recursive Function

- I. It is a **recursive** function – a function that calls itself.
- II. It will keep on calling itself, with different parameters, until a terminating condition is met.
- III. **It's like writing a loop. You can also write a loop to do something over and over again, until some exiting condition is met.**
- IV. **Resolution the recursion mathematic function problem**

ลักษณะการเขียนโปรแกรมแบบ Recursion

Direct Recursion

```
void f()  
{  
    . . .  
    f();  
    . . .  
}
```

(a)

Indirect Recursion

```
void a()  
{  
    . . .  
    b();  
    . . .  
}
```

```
void b()  
{  
    . . .  
    c();  
    . . .  
}
```

```
void c()  
{  
    . . .  
    d();  
    . . .  
}
```

```
void d()  
{  
    . . .  
    a();  
    . . .  
}
```

(b)

Figure 1. Direct and indirect recursion.

Recursive function pattern

main.pas

```
1 program RecursiveProcedureTemplate;
2
3 procedure RecursiveProc(n: Integer);
4 begin
5     if n <= 0 then
6     begin
7         // Base case: stop recursion
8         WriteLn('Reached base case with n = ', n);
9         Exit;
10    end;
11
12    // Do something before recursive call
13    WriteLn('Before recursive call, n = ', n);
14
15    // Recursive call with modified parameter
16    RecursiveProc(n - 1);
17
18    // Do something after recursive call
19    WriteLn('After recursive call, n = ', n);
20 end;
21
22 begin
23     RecursiveProc(3);
24 end.
```

Step recursion

(2)

Call its self function

(1)

Linking a.out

```
24 lines compiled, 0.0 sec
Before recursive call, n = 3
Before recursive call, n = 2
Before recursive call, n = 1
Reached base case with n = 0
After recursive call, n = 1
After recursive call, n = 2
After recursive call, n = 3
```

recvTmple.pas

fwNum.pas

```
1  program ForwardDigits;
2
3  procedure Forward(number: Integer);
4  begin
5      if number <> 0 then
6      begin
7          Forward(number div 10);
8          Write(number mod 10);
9      end;
10 end;
11
12 var
13     n: Integer;
14 begin
15     Write('Enter number: ');
16     ReadLn(n);
17
18     Forward(n);
19     WriteLn;
20 end.
21
```

fwNum.pas

4. forward(0)
5. cout << 1
3. forward(1)
6. cout << 2
2. forward(12)
7. cout << 3
1. forward(123)

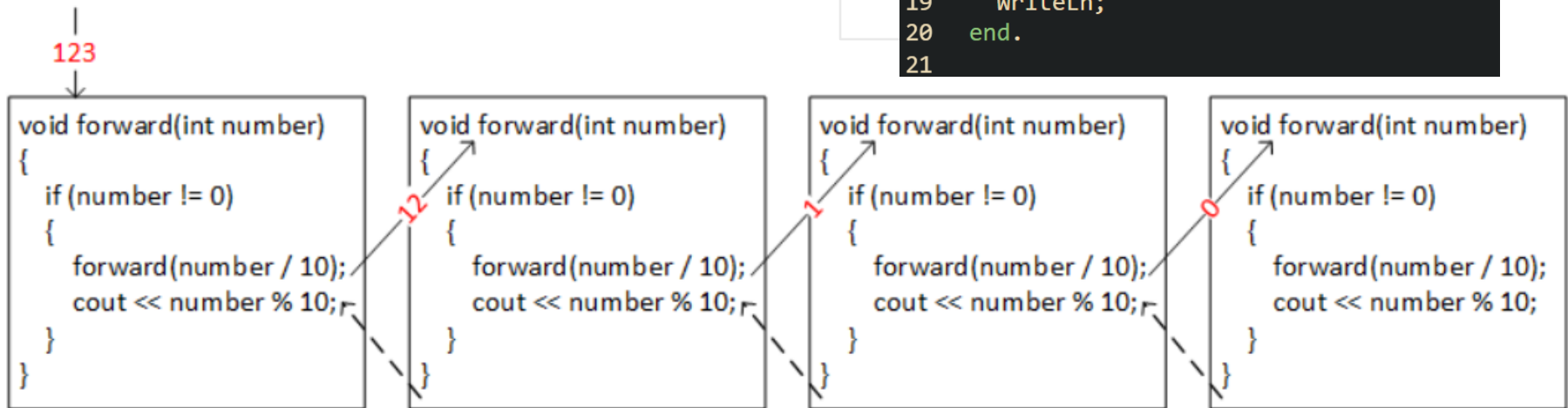
number: 0	4 th
number: 1	3 rd
number: 12	2 nd
number: 123	1 st

(a)

```

1  program ForwardDigits;
2
3  procedure Forward(number: Integer);
4  begin
5      if number <> 0 then
6      begin
7          Forward(number div 10);
8          Write(number mod 10);
9      end;
10 end;
11
12 var
13     n: Integer;
14 begin
15     Write('Enter number: ');
16     ReadLn(n);
17
18     Forward(n);
19     WriteLn;
20 end.
21

```



(c)



revNum.pas

```
1  program reverseDigits;
2
3  procedure reverse(number: Integer);
4  begin
5      if number <> 0 then
6      begin
7          Write(number mod 10);
8          reverse(number div 10);
9      end;
10 end;
11
12 var
13     n: Integer;
14 begin
15     Write('Enter number: ');
16     ReadLn(n);
17
18     reverse(n);
19     WriteLn;
20 end.
21
```



```
6. cout << 1
7. reverse(0)

4. cout << 2
5. reverse(1)

2. cout << 3
3. reverse(12)

1. reverse(123)
```

number: 0
number: 1
number: 12
number: 123

(b)

123

```
procedure reverse
  (number: Integer);
begin
  if number <> 0 then
  begin
    Write(number mod 10);
    reverse(number div 10);
  end;
end;
```

12

```
procedure reverse
  (number: Integer);
begin
  if number <> 0 then
  begin
    Write(number mod 10);
    reverse(number div 10);
  end;
end;
```

1

```
procedure reverse
  (number: Integer);
begin
  if number <> 0 then
  begin
    Write(number mod 10);
    reverse(number div 10);
  end;
end;
```

0

```
procedure reverse
  (number: Integer);
begin
  if number <> 0 then
  begin
    Write(number mod 10);
    reverse(number div 10);
  end;
end;
```



หลักการเขียนโปรแกรม Recursive

มี 2 องค์ประกอบ recursive Math equation

1. Base case (จุดหยุด)

2. Recursive case (อ้างอิงตัวเอง)

Factorial:

$$0! = 1$$

เมื่อ $n=0$

$$n! = n \cdot (n - 1)!$$

เมื่อ $n>0$

$$4! = 4 \times 3 \times 2 \times 1$$

Apirak :

Recursive Function template

1. function f(problem):
2. if base_case(problem):
3. return base_answer
4. else:
5. smaller_problem = reduce(problem)
6. sub_result = f(smaller_problem)
7. return combine(problem, sub_result)

Base case (stop condition)

มี 2 องค์ประกอบ

1. Base case (จุดหยุด)

2. Recursive case (อ้างอิงตัวเอง)

Factorial:

$$0! = 1 \quad \text{เมื่อ } n=0$$

$$n! = n \cdot (n - 1)! \quad \text{เมื่อ } n>0$$

$$4! = 4 \times 3 \times 2 \times 1$$

Recursive call

factRecv.pas

```
1 program FactorialRecursive;
2
3 function Fact (n: Integer) : Integer;
4 var soll, solution: Integer;
5 begin
6   if n = 0 then
7     exit(1) { base case: 0! = 1 }
8   else
9     begin
10      soll := Fact (n - 1); { recursive call }
11      solution := n * soll;
12      exit(solution);
13    end;
14 end;
15
16 var
17   n: Integer;
18 begin
19   Write('Enter n: ');
20   ReadLn(n);
21
22   WriteLn(n, '! = ', Fact(n));
23 end.
24
```

$$n! = n \cdot (n - 1)! \quad (n > 0)$$

$$0! = 1$$

recursive function
Base case

```
public class Recurse1
```

```
{
```

```
    public static void main( String[] args )
```

```
    {
```

```
        double r;
```

```
        r = factorial( 3 ); ← point A
```

```
    }
```

```
    public static int factorial ( int n )
```

```
    {
```

```
        int sol1, solution;
```

```
        if ( n == 0 )
```

```
        {
```

```
            return 1;
```

```
        }
```

```
        else
```

```
        {
```

```
            sol1 = factorial(n-1); ← point B
```

```
            solution = n * sol1;
```

```
            return( solution );
```

```
        }
```

```
    }
```

```
}
```

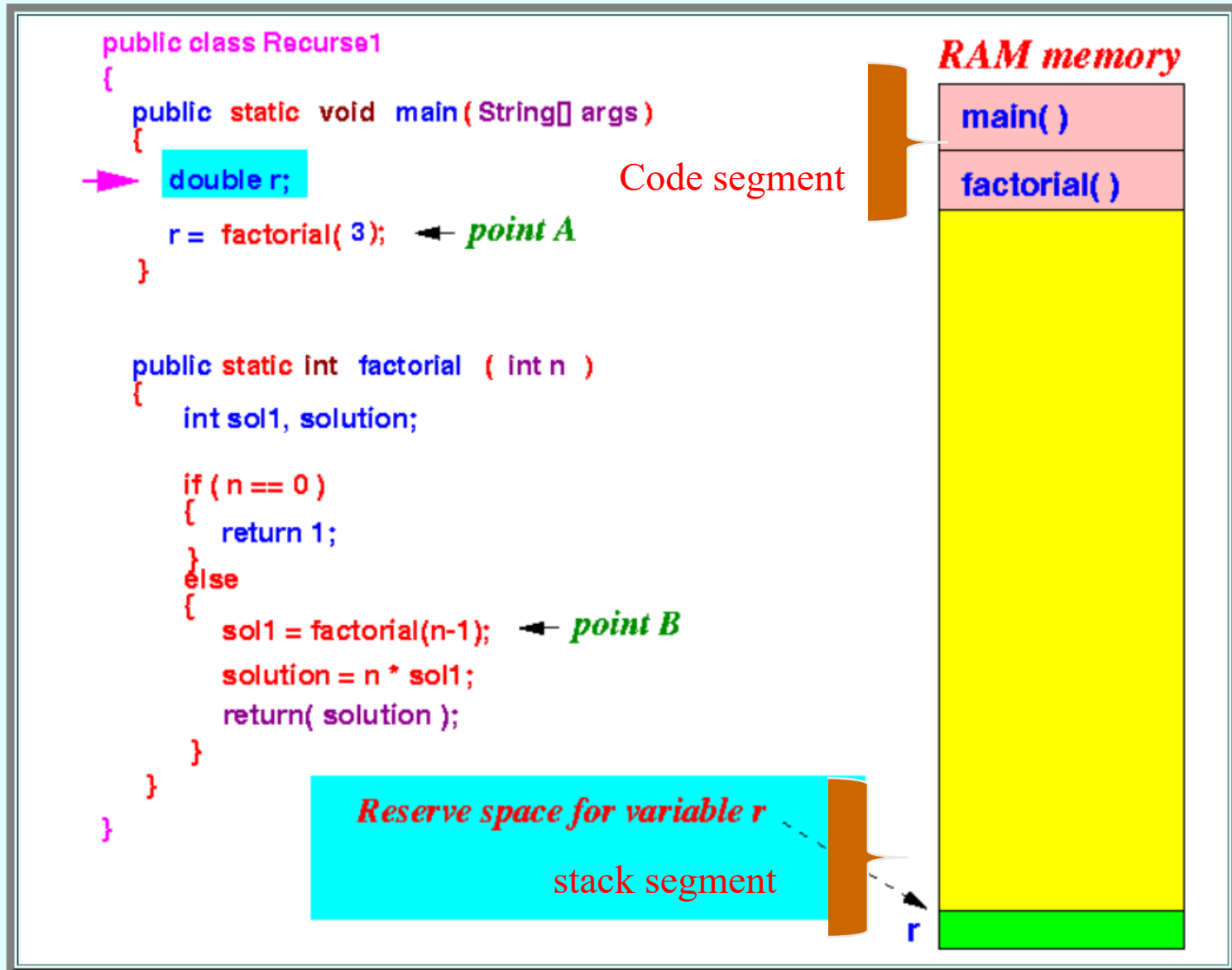
Code segment

RAM memory

main()

factorial()

- Execution starts in `main()`:



Reserves space for local variable *r* of `main()`

- main() calls factorial(3) - passes parameter 3

```
public class Recurse1
{
    public static void main (String[] args)
    {
        double r;
        r = factorial( 3);
    }
```

← point A

```
public static int factorial ( int n )
{
    int sol1, solution;

    if ( n == 0 )
    {
        return 1;
    }
    else
    {
        sol1 = factorial(n-1);
        solution = n * sol1;
        return( solution );
    }
}
```

← point B

return
n
r

RAM memory

main()

factorial()

point A

3

- `factorial(3)` calls `factorial(n-1 (2))` - passes parameter 2

```
public class Recurse01
{
    public static void main( String[] args )
    {
        double r;
        r = factorial( 3); ← point A
    }
}
```

```
public static int factorial ( int n )
{
    int sol1, solution;

    if ( n == 0 )
    {
        return 1;
    }
    else
    {
        sol1 = factorial(n-1); ← point B
        solution = n * sol1;
        return( solution );
    }
}
```

RAM memory

main()

factorial()

return
n
solution
sol1
return
n
r

point B

2

point A

3

Notice that a *new copy* of the *parameter variable n* is *created*.

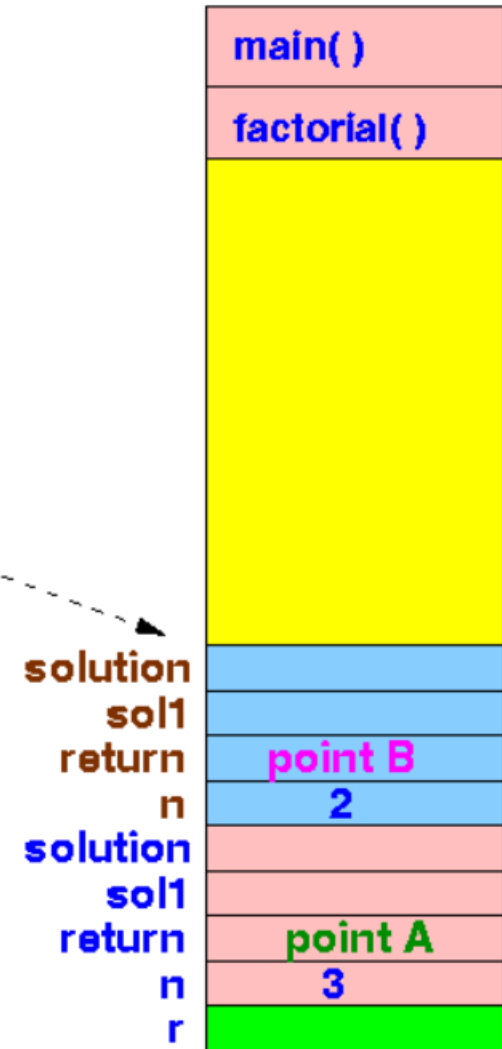
Apurak :

- `factorial(2)` will create the **local variables**:

```
public class Recurse1
{
    public static void main( String[] args )
    {
        double r;
        r = factorial( 3); ← point A
    }
}
```

```
public static int factorial ( int n )
{
    int sol1, solution;
    if ( n == 0 )
    {
        return 1;
    }
    else
    {
        sol1 = factorial(n-1); ← point B
        solution = n * sol1;
        return( solution );
    }
}
```

RAM memory



Notice that **each factorial method** will use **it's own parameter and local variables !!!**

-

```

public static int factorial ( int n )
{
    int sol1, solution;

    if ( n == 0 )
    {
        return 1;
    }
    else
    {
        sol1 = factorial(n-1);
        solution = n * sol1;
        return( solution );
    }
}

```

main()
factorial()
point B 1
point B 2
point A 3

Apirak :

- `factorial(1)` calls `factorial(n-1 (0))` - passes parameter 0

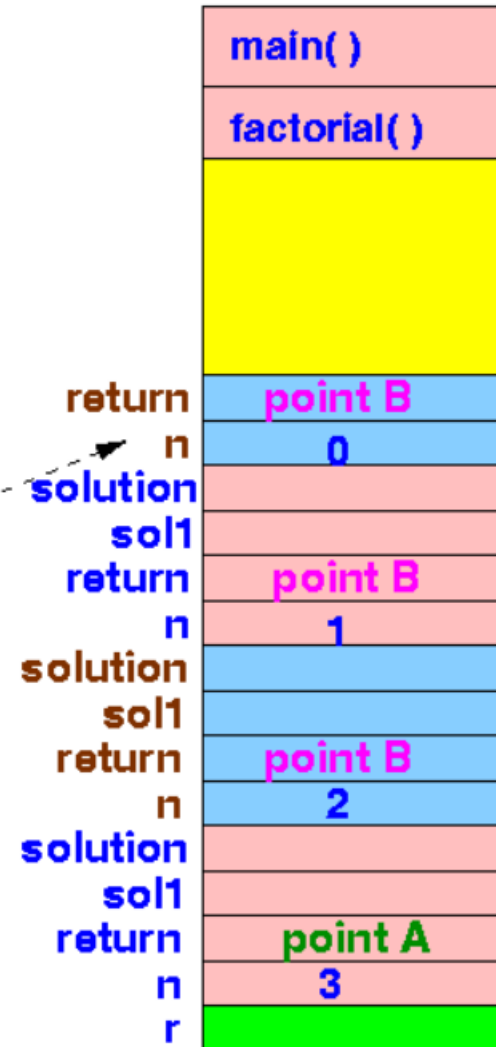
```
public class Recurse1
{
    public static void main( String[] args )
    {
        double r;

        r = factorial( 3); ← point A
    }
}
```

```
public static int factorial ( int n )
{
    int sol1, solution;

    if ( n == 0 )
    {
        return 1;
    }
    else
    {
        ← point B sol1 = factorial(n-1);
        solution = n * sol1;
        return( solution );
    }
}
```

RAM memory

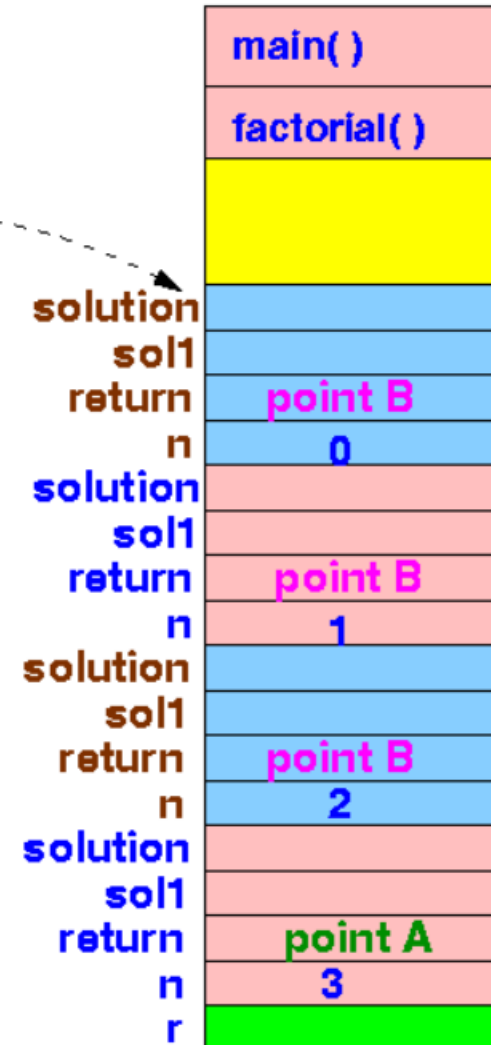


- `factorial(0)` will create the **local variables**:

```
public class Recurse1
{
    public static void main( String[] args )
    {
        double r;
        r = factorial( 3); ← point A
    }
}
```

```
public static int factorial ( int n )
{
    int sol1, solution;
    if ( n == 0 )
    {
        return 1;
    }
    else
    {
        sol1 = factorial(n-1); ← point B
        solution = n * sol1;
        return( solution );
    }
}
```

RAM memory



- factorial(0) will *return the value 1*:

```

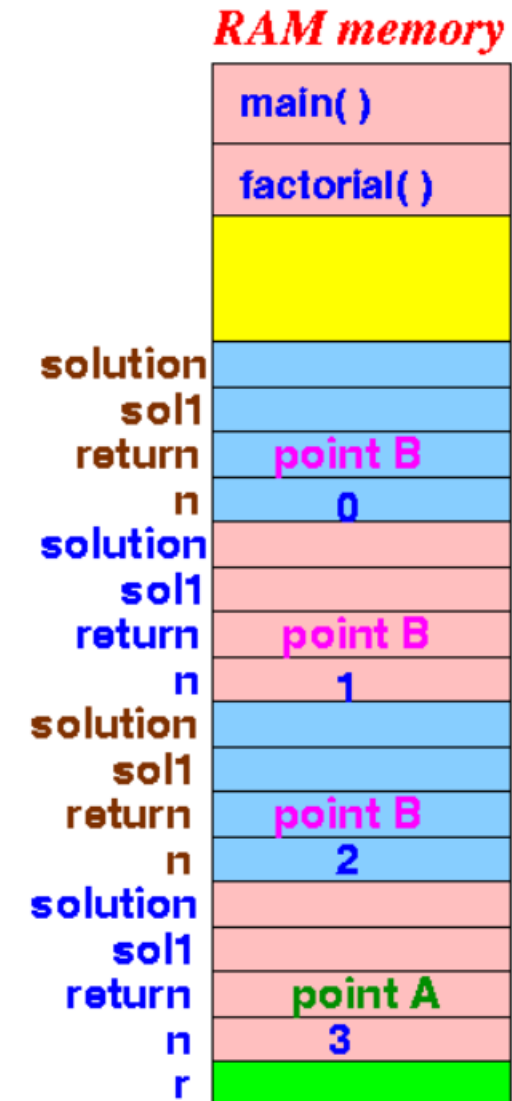
public class Recurse1
{
    public static void main( String[] args )
    {
        double r;

        r = factorial( 3); ← point A
    }

    public static int factorial ( Int n )
    {
        int sol1, solution;

        if ( n == 0 )
        {
            return 1;
        }
        else
        {
            sol1 = factorial(n-1); ← point B
            solution = n * sol1;
            return( solution );
        }
    }
}

```



- `factorial(1)` will *return the value 1*:

```

public class Recurse1
{
    public static void main( String[] args )
    {
        double r;

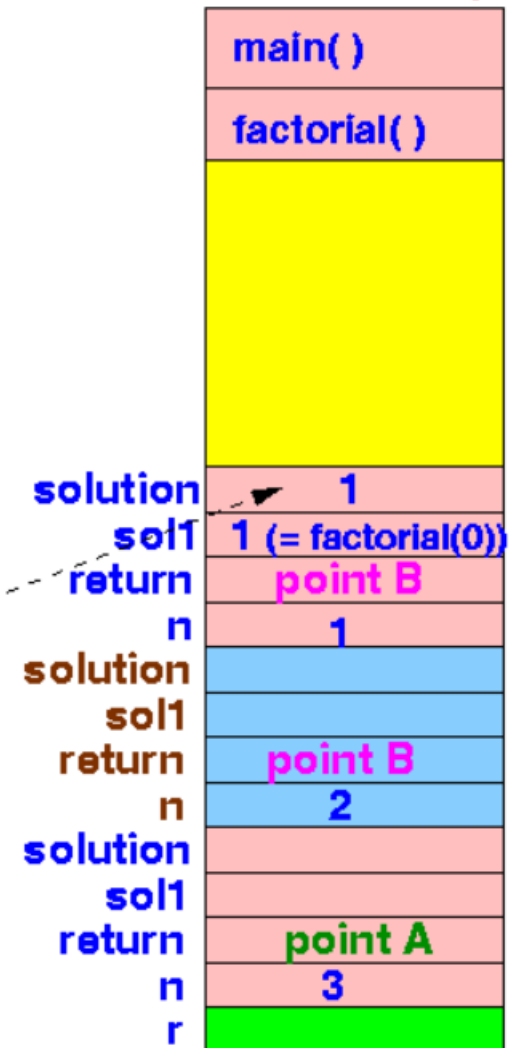
        r = factorial( 3 ); ← point A
    }

    public static int factorial ( int n )
    {
        int sol1, solution;

        if ( n == 0 )
        {
            return 1;
        }
        else
        {
            sol1 = factorial(n-1); ← point B
            solution = n * sol1;
            return( solution );
        }
    }
}

```

RAM memory



`factorial(1)` will now **terminate**.

- Execution **returns** to *point B* in `factorial(2)`:

```

public class Recurse1
{
    public static void main( String[] args )
    {
        double r;

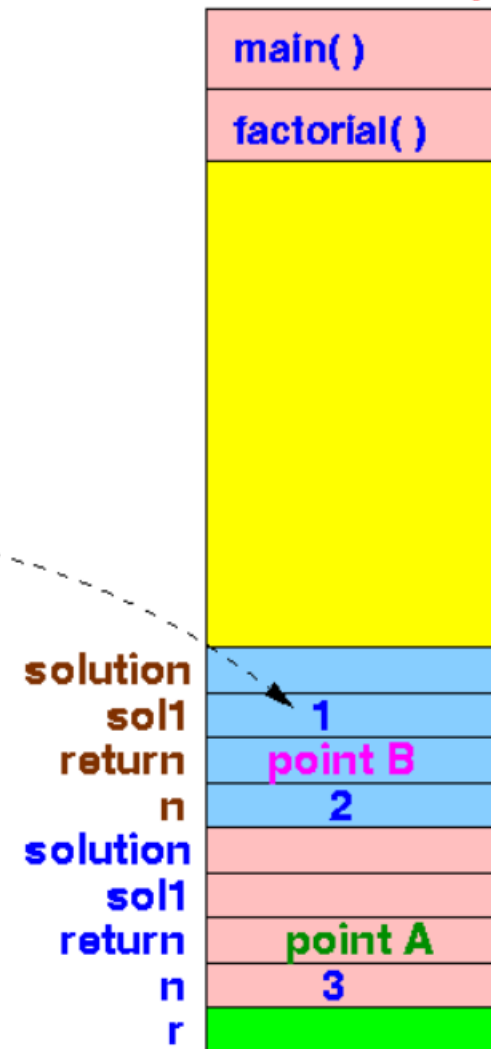
        r = factorial( 3 ); ← point A
    }

    public static int factorial ( int n )
    {
        int sol1, solution;

        if ( n == 0 )
        {
            return 1;
        }
        else
        {
            ← point B
            sol1 = factorial(n-1);
            solution = n * sol1;
            return( solution );
        }
    }
}

```

RAM memory



`factorial(2)` will use the solution `sol1 (= 1)` and **return** `solution = 2 × 1 = 2`

- `factorial(2)` will *return the value 2*:

```
public class Recurse1
{
    public static void main( String[] args )
    {
        double r;

        r = factorial( 3);  ← point A
    }

    public static int factorial ( int n )
    {
        int sol1, solution;

        if ( n == 0 )
        {
            return 1;
        }
        else
        {
            sol1 = factorial(n-1);  ← point
            solution = n * sol1;
            return( solution );
        }
    }
}
```

RAM memory

	main()
	factorial()
solution sol1 return n solution sol1 return n r	2 (= 2x1) 1 point B 2 point A 3

`factorial(2)` will now **terminate**.

- Execution returns to *point B* in `factorial(3)`:

```

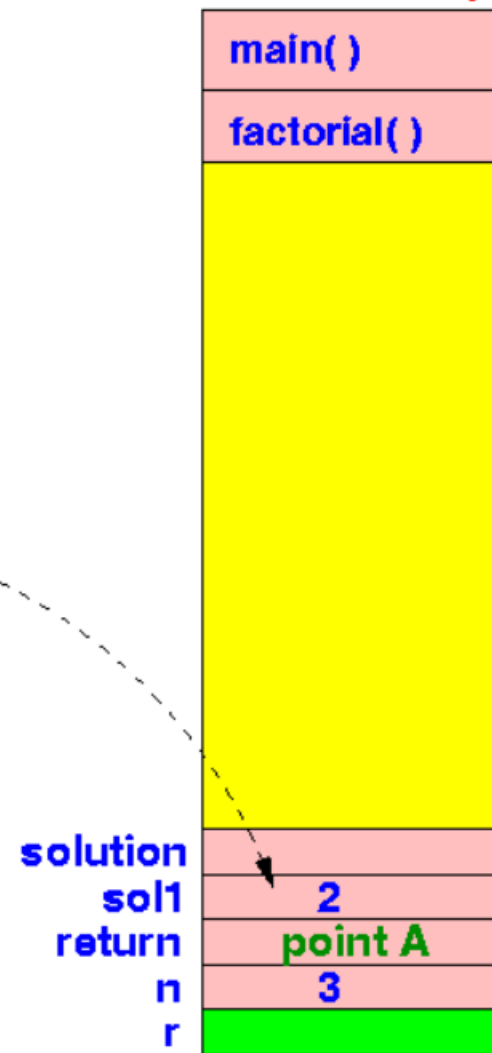
public class Recurse1
{
    public static void main( String[] args )
    {
        double r;
        r = factorial( 3); ← point A
    }

    public static int factorial ( int n )
    {
        int sol1, solution;

        if ( n == 0 )
        {
            return 1;
        }
        else
        {
            ← point B
            sol1 = factorial(n-1);
            solution = n * sol1;
            return( solution );
        }
    }
}

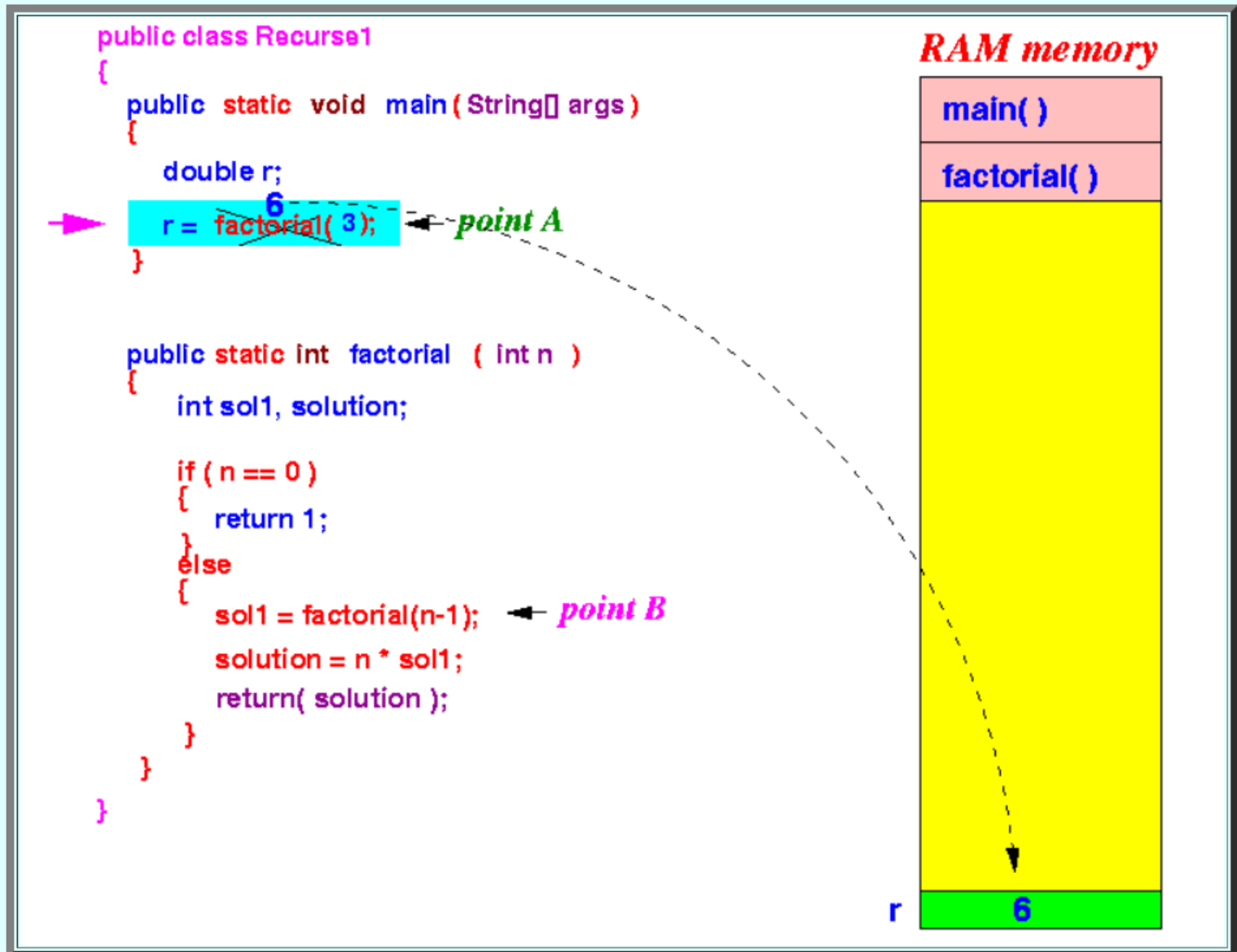
```

RAM memory



`factorial(3)` will use the solution `sol1 (= 2)` and return `solution = 3 × 2 = 6`

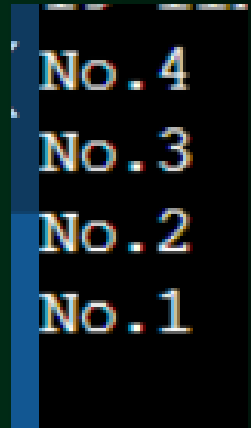
- Execution returns to *point A* in `main()`:



We are **finally** done !!!



ตย1.จงแปลงให้เป็นโปรแกรม recursive



$\text{Print}(i) = \begin{cases} \text{Stop , เมื่อ } i \leq 0 \\ \text{write}(i), \text{Print}(i-1), \text{ เมื่อ } i > 0 \end{cases}$

มี 2 องค์ประกอบ

1.Base case (จุดหยุด)

2.Recursive case (อ้างอิงตัวเอง)

Factorial:

$0! = 1$ เมื่อ $n=0$

$n! = n \cdot (n - 1)!$ เมื่อ $n>0$

$4! = 4 \times 3 \times 2 \times 1$

```

1  program ForLoopExample1;
2
3  procedure print( i:Integer );
4  begin
5      if( i<=0) then
6      begin
7          exit();
8      end;
9      WriteLn('No.', i);
10     print( i-1 );
11 end;
12
13 begin
14     print(4);
15 end.

```

Factorial

- function $f(\text{problem})$:
- if base_case(problem):
- return base_answer
- else:
- smaller_problem = reduce(problem)
- sub_result = $f(\text{smaller_problem})$
- return combine(problem, sub_result)

Base case

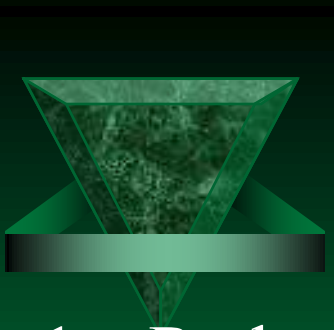
Print(i) = { Stop , เมื่อ $i \leq 0$
 write(i), Print(i-1), เมื่อ $i > 0$

แนวคิดการเขียน โปรแกรม/แปลงโปรแกรมให้การทำงานซ้ำ



แนวคิด

1. Reducing toward a base case (คล้าย **Factorial**)
2. Progressing state until a termination condition



1. Reducing toward a base case (คล้าย **Factorial**)

The first Caller จะส่งค่า max ไป หรือจำนวนครั้งสูงสุดที่ต้องการให้ทำซ้ำ
`Print(5)` => The first caller

`Print( i ) =` $\left\{ \begin{array}{l} \text{Stop , เมื่อ } i \leq 0 \\ \text{Print}(i-1), \text{write}(i) \end{array} \right.$



2. Progressing state until a termination condition

The first Caller จะส่งเริ่มต้นการทำซ้ำ เพื่อเป็นเงื่อนไขของการทำซ้ำ

`Print(1)` $\Rightarrow$ The first caller

`Print( i ) =` $\left\{ \begin{array}{l} \text{Stop , เมื่อ } i > 5 \\ \text{write(i) ; Print(i+1)} \end{array} \right.$

ตย2. จงแปลงโปรแกรมให้เป็น recursive

```
1
2 program ForLoopExample;
3   var i: Integer;
4 begin
5   for i := 1 to 3 do
6   begin
7     WriteLn('Line No.', i);
8   end;
9 end.
```

```
19 lines comp.
Line No.1
Line No.2
Line No.3
```

Reducing toward a base case

```
1
2 program ForLoopExample;
3   var i: Integer;
4   begin
5     for i := 1 to 3 do
6     begin
7       WriteLn('Line No.', i);
8     end;
9   end.
10
```

มี 2 องค์ประกอบ

1. Base case (จุดหยุด)

2. Recursive case (อ้างอิงตัวเอง)

Factorial:

$$0! = 1$$

เมื่อ $n=0$

$$n! = n \cdot (n - 1)!$$

เมื่อ $n>0$

$$4! = 4 \times 3 \times 2 \times 1$$

$\text{Print}(i) = \begin{cases} \text{Stop , เมื่อ } i \leq 0 \\ \text{Print}(i-1), \text{write}(i) \end{cases}$

Reducing toward a base case

main.pas

```
1 program ForLoopExample1;
2
3 procedure print( i:Integer );
4 begin
5     if( i<=0) then
6     begin
7         exit();
8     end;
9     print( i-1 );
10    WriteLn('Line No.', i);
11 end;
12
13 begin
14     print(3);
15 end.
16
```

Recursive Function template

1. function f(problem):
2. if base_case(problem):
3. return base_answer
4. else:
5. smaller_problem = reduce(problem)
6. sub_result = f(smaller_problem)
7. return combine(problem, sub_result)

1. Base case (จุดหยุด)
2. Recursive case (เรียกซ้ำ)

Factorial:

$$0! = 1$$

$$n! = n \cdot (n-1)!$$

$$4! = 4 \times 3 \times 2 \times 1$$

Base case (stop)

Recursion

Print(i) = { Stop , เมื่อ $i \leq 0$
Print(i-1), write(i)

Progressing state until a termination condition

```
1
2 program ForLoopExample;
3   var i: Integer;
4 begin
5   for i := 1 to 3 do
6   begin
7     WriteLn('Line No.', i);
8   end;
9 end.
```

มี 2 องค์ประกอบ

1.Base case (จุดหยุด)

2.Recursive case (อ้างอิงตัวเอง)

Factorial:

$0! = 1$ เมื่อ $n=0$

$n! = n \cdot (n - 1)!$ เมื่อ $n>0$

$4! = 4 \times 3 \times 2 \times 1$

```
19 lines comp.
Line No.1
Line No.2
Line No.3
```

Print(i) = { Stop , เมื่อ $i > 3$
write(i),Print(**i+1**), โดย i เริ่มที่ 1

Caller : Print(1)

Progressing state until a termination condition

template

```
1 program ForLoopExample1;  
2  
3 procedure print( i:Integer );  
4 begin  
5     if( i>3) then  
6     begin  
7         exit();  
8     end;  
9     WriteLn('Line No.', i);  
10    print( i+1 );  
11 end;  
12  
13 begin  
14     print(1);  
15 end.
```

1. function f(problem):
2. if base_case(problem):
3. return base_answer
4. else:
5. smaller_problem = reduce(problem)
6. sub_result = f(smaller_problem)
7. return combine(problem, sub_result)

Base case

recvLoopV1.pas

Print(i) = { Stop , เมื่อ i > 3
write(i), Print(i+1), โดย i เริ่มที่ 1



Progressing state until a termination condition

กำหนด n เพื่อควบคุม base case

$\text{Print}(i, n) = \begin{cases} \text{Stop, เมื่อ } i > n \\ \text{write}(i), \text{Print}(i+1, n), \text{ โดย } i \text{ เริ่มที่ } 1 \end{cases}$

The First Caller : $\text{Print}(1, 4)$

$\text{Print}(i) = \begin{cases} \text{Stop, เมื่อ } i > 3 \\ \text{write}(i), \text{Print}(i+1), \text{ โดย } i \text{ เริ่มที่ } 1 \end{cases}$

Line No.3



rewrite

ตย3. จงแปลงโปรแกรมให้เป็น recursive



```
main.pas
1  program SumExample;
2
3  var
4      s, i: Integer;
5
6  begin
7      s := 0;
8      for i := 1 to 5 do
9          begin
10             s := s + i;
11          end;
12      WriteLn('sum = ', s);
13  end.
14
```

Progressing state until a termination condition

```
1 program SumExample;  
2 |  
3 var  
4   s, i: Integer;  
5  
6 begin  
7   s := 0;  
8   for i := 1 to 5 do  
9     begin  
10      s := s + i;  
11    end;  
12    WriteLn('sum = ', s);  
13  end.  
14
```

การ call: Sum(5)

recvSum5v2.pas

$$\text{sum}(i) = \begin{cases} 0 & \text{เมื่อ } i \leq 0 \\ i + \text{Sum}(i-1) \end{cases}$$

main.pas

Reducing toward a base case

```
1 program SumExample;  
2  
3 function sum(i:integer): integer;  
4 var s:integer;  
5 begin  
6   if( i <=0 ) then  
7     exit(0 );  
8   exit ( i+sum(i-1) );  
9 end;  
10  
11 var  
12   s : Integer;  
13 begin  
14   s:=sum(5);  
15   WriteLn('sum = ', s);  
16 end.  
17
```

การ call: Sum(5)

recvSum5v2.pas

$$\text{sum}(i) = \begin{cases} 0 & \text{เมื่อ } i \leq 0 \\ i + \text{Sum}(i-1) \end{cases}$$

```

main.pas
1 program SumExample;
2
3 var
4   s, i: Integer;
5
6 begin
7   s := 0;
8   for i := 1 to 5 do
9     begin
10      s := s + i;
11    end;
12   WriteLn('sum = ', s);
13 end.
14

```

มองแบบ loop i เริ่มที่ 1

มี 2 องค์ประกอบ

1. Base case (จุดหยุด)
2. Recursive case (อ้างอิงตัวเอง)

Factorial:

$0! = 1$ เมื่อ $n=0$

$n! = n \cdot (n - 1)!$ เมื่อ $n > 0$

$4! = 4 \times 3 \times 2 \times 1$

$\text{sum}(i) = \begin{cases} 0 & \text{เมื่อ } i > 5 \\ i + \text{Sum}(i+1) & \text{เมื่อ } i \text{ เริ่มที่ } 1 \end{cases}$

การ call: Sum(1)



Reducing toward a base case

sum(1 ,5)

$$\text{sum}(i ,n) = \begin{cases} 0 & \text{เมื่อ } i > n \\ i + \text{Sum}(i+1,n) \end{cases}$$

Progressing state until a termination condition

```
1 program SumExample;  
2  
3 function sum(i:integer): integer;  
4 var s:integer;  
5 begin  
6   if( i >5 ) then  
7     exit(0 );  
8     exit ( i+sum(i+1) );  
9 end;  
10  
11 var  
12   s : Integer;  
13 begin  
14   s:=sum(1);  
15   WriteLn('sum = ', s);  
16 end.
```

มองแบบ loop i เริ่มที่ 1

การ call: Sum(1)

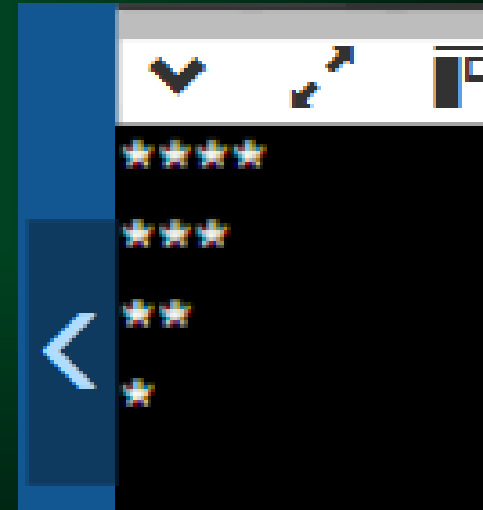
$$\text{sum}(i) = \begin{cases} 0 & \text{เมื่อ } i > 5 \\ i + \text{Sum}(i+1) & \text{เมื่อ } i \text{ เริ่มที่ } 1 \end{cases}$$

ตย3. จงแปลงโปรแกรมให้เป็น recursive

main.pas

```
1 program StarPattern;
2
3 var
4     n, j: integer;
5
6 begin
7
8     n := 4;
9     while n > 0 do
10 begin
11     for j := 0 to n - 1 do
12 begin
13     write('*');
14 end;
15 writeln;
16     n := n - 1;
17 end;
18
19 end.
20
```

loopPrnStart.pas



Reducing toward a base case

main.pas

```
1 program StarPattern;
2
3 var
4   n, j: integer;
5
6 begin
7
8   n := 4;
9   while n > 0 do
10    begin
11      for j := 0 to n - 1 do
12        begin
13          write('*');
14        end;
15      writeln;
16      n := n - 1;
17    end;
18
19 end.
20
```

print(i) =

{ stop เมื่อ $i \leq 0$
Write('*',
print(i-1)

แบบ max: print(5)

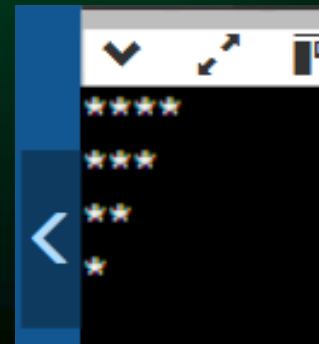
Reducing toward a base case

main.pas

```
1 program StarPatternV1;
2
3 procedure PrnStar(n:integer);
4 begin
5   if( n <=0) then
6     exit();
7     write('*');
8     PrnStar(n-1);
9 end;
10
11 var
12   n, j: integer;
13
14 begin
15
16   n := 4;
17   while n > 0 do
18   begin
19     PrnStar(n);
20     writeln;
21     n := n - 1;
22   end;
23
24 end.
25
```

rcvPrnStartV1.pas

makprint(i) = { stop เมื่อ $i \leq 0$
prnStart(i),
makprint(i-1)



Reducing toward a base case

main.pas

```
1 program StarPattern;
2
3 var
4   n, j: integer;
5
6 procedure PrnStar(n:integer);
7 begin
8   if( n <=0) then
9     exit();
10  write('*');
11  PrnStar(n-1);
12 end;
13
14 procedure makeStar(n:integer);
15 begin
16   if( n <=0) then
17     exit();
18   PrnStar(n);
19   writeln;
20   makeStar(n-1);
21 end;
22
23 begin
24   makeStar(4);
25 end.
26
```

rcvPrnStartV2.pas

makprint(i) = { stop เมื่อ $i \leq 0$
prnStart(i) ,
makprint(i-1)


```

1  program StarPatternV3;
2  procedure print (j, n:integer);
3  begin
4      if (j > n) then
5          exit();
6          write('*');
7          print(j+1, n);
8  end;
9
10 var
11     n:integer;
12
13 begin
14     ..
15     n := 4;
16     while n > 0 do
17     begin
18         print(1, n);
19         writeln;
20         n := n - 1;
21     end;
22     ..
23 end.

```

$\text{print}(i, n) = \begin{cases} \text{stop} & \text{เมื่อ } i > n \\ \text{Write('*') ,} \\ \text{print}(i+1, n) \end{cases}$

แบบ loop: print(1,5)



HOMEWORK

จงแปลงโปรแกรมต่อไปนี้ให้เป็น recursive function

```
Linking a.out
18 lines comp
sum = 140
```

main.pas

```
1
2 program SumDowhileLoop;
3
4 var
5     sum, y: integer;
6
7 begin
8
9     sum := 0;
10    y := 50;
11    repeat
12        sum := sum + y;
13        y := y - 10;
14    until y <= 10;
15
16    writeln('sum = ', sum);
17
18 end.
19
```

```

1
2 program SumDowhileLoop;
3
4 var
5     sum, y: integer;
6
7 begin
8
9     sum := 0;
10    y := 50;
11    repeat
12        sum := sum + y;
13        y := y - 10;
14    until y <= 10;
15
16    writeln('sum = ', sum);
17
18 end.
19

```

$$\text{sum}(y) = \begin{cases} 0 & \text{เมื่อ } y \leq 10 \\ y + \text{Sum}(y-10) & \end{cases}$$

$$\text{sum}(i) = \begin{cases} 0 & \text{เมื่อ } i > 5 \\ i + \text{Sum}(i+1) & \text{เมื่อ } i \text{ เริ่มที่ } 1 \end{cases}$$

```

1
2
3  program SumDoWhileLoop;
4  function sum(y:integer):integer;
5  begin
6      if(y<=10) then
7          exit(0);
8      ..
9      exit(y+sum(y-10))
10 end;
11
12 begin
13     writeln('sum.=.', sum(50));
14 end.
15

```

prob1.pas

$$\text{sum}(y) = \begin{cases} 0 & \text{เมื่อ } y \leq 10 \\ y + \text{Sum}(y-10) \end{cases}$$



BinConv.pas

จงเขียนโปรแกรมแปลงเลขฐาน 10 เป็น 2

2	221	เลข	1 (LSB)
2	110	เลข	0
2	55	เลข	1
2	27	เลข	1
2	13	เลข	1
2	6	เลข	0
2	3	เลข	1
2	1	เลข	1
	0		

$$\therefore (221)_{10} = (11011101)_2$$

main.pas

```
1
2 program BinaryConverter;
3
4 var
5     x, b: integer;
6
7 begin
8
9     write('pls, enter data (int): ');
10    readln(x);
11
12    while x <> 0 do
13    begin
14        b := x mod 2;
15        x := x div 2;
16        writeln(b);
17    end;
18
19 end.
```

```
< 24 lines compiled, 0.0 sec
pls, enter data (int): 5
1
0
1
```

BinConvV2.pas

Solution

2	221	เลข	1 (LSB)
2	110	เลข	0
2	55	เลข	1
2	27	เลข	1
2	13	เลข	1
2	6	เลข	0
2	3	เลข	1
2	1	เลข	1
	0	เลข	1 (MSB)

$\therefore (221)_{10} = (11011101)_2$

$\text{cov2bin}(y) = \left\{ \begin{array}{l} \text{stop} \quad \text{เมื่อ } y \neq 0 \\ \text{Write}(y \bmod 2) \\ \text{cov2bin}(y / 2) \end{array} \right.$

BinConvV2.pas

Solution

```
1  program BinaryConverter;
2
3  procedure cov2bin(x:integer);
4  var b: integer;
5  begin
6    if (x <> 0) then
7    begin
8      b := x mod 2;
9      x := x div 2;
10     writeln(b);
11     cov2bin(x);
12   end;
13 end;
14
15 var x:integer;
16 begin
17   ..
18   write('pls, enter data (int): ');
19   // readln(x);
20   writeln;
21   cov2bin(5);
22 end.
```

$\text{cov2bin}(y) = \begin{cases} \text{stop} & \text{เมื่อ } y \neq 0 \\ \text{Write}(y \bmod 2) \\ \text{cov2bin}(y / 2) \end{cases}$

โปรแกรมแปลงฐาน 10 เป็นฐาน 2 และ 16(1)

main.pas

```
1 program DecToHexConverter;
2 type
3     binArray = array[1..19] of integer;
4
5 procedure Dec2Bin(x: integer; var bin: binArray ; i: integer; var n: integer);
6 var
7     b: integer;
8 begin
9     if x <> 0 then
10 begin
11     b := x mod 2;
12     x := x div 2;
13     bin[i] := b;
14     n := i + 1;
15     Dec2Bin(x, bin, i + 1, n);
16 end;
17 end;
18
19 procedure Dec2Hex(x: integer; var bin: binArray ; i: integer; var n: integer);
20 var
21     b: integer;
22 begin
23     if x <> 0 then
24 begin
25     b := x mod 16;
26     x := x div 16;
27     bin[i] := b;
28     n := i + 1;
29     Dec2Hex(x, bin, i + 1, n);
30 end;
31 end;
```

dec2Hex.pas

โปรแกรมแปลงฐาน 10 เป็นฐาน 2 และ 16 (2)

```
32
33 var
34     x, i, j, n: integer;
35     bin: binArray;
36 begin
37
38     write('pls, enter data (int): ');
39     //readln(x);
40     x := 254; writeln;
41     n := 0;
42     Dec2Hex(x, bin, 0, n);
43
44     for j := n - 1 downto 0 do
45     begin
46         case bin[j] of
47             10: write('A');
48             11: write('B');
49             12: write('C');
50             13: write('D');
51             14: write('E');
52             15: write('F');
53         else
54             write(bin[j]);
55         end;
56     end;
57
58     writeln;
59
60 end.
```

bin2Hex.pas โปรแกรมแปลงฐาน 2 เป็นฐาน 10

```
1  program BinToDecConverter;
2
3  var
4      bin: string;
5      dec, p, d, i: integer;
6  begin
7
8      write('Enter binary number: ');
9      //readln(bin);
10     bin := '1010101';
11     dec := 0;
12     p := 1;
13
14     for i := length(bin) downto 1 do
15     begin
16         case bin[i] of
17             '0': d := 0;
18             '1': d := 1;
19         end;
20         dec := dec + p * d;
21         p := p * 2;
22     end;
23
24     writeln('Decimal = ', dec);
25     readln;
26 end.
```



ข้อ 6 กำหนดฟังก์ชัน $CV(N)$ เมื่อ $N \geq 1$

$$CV(1) = 2$$

$$CV(2) = 5$$

$$CV(3) = CV(1) * 2 + 1 = 5$$

$$CV(4) = CV(2) * 3 - 5 = 10$$

$$CV(5) = CV(3) * 2 + 1 = 11$$

$$CV(6) = CV(4) * 3 - 5 = 25$$

จงเขียนโปรแกรม C++ โดยใช้ recursive subprogram เพื่อทำการคำนวณหาค่า $CV(N)$ เมื่อ N คือ INPUT DATA

ข้อ 5 พิจารณาผลบวกของข้อมูลต่อไปนี้

$$(1/10) + (2/10) + (3/10) + (4/10) + \dots + (n/10)$$

สมมติว่า n เป็นเลขจำนวนเต็มบวกใดๆ

ถ้า $n=1$ ผลคือ $1/10$

$n=2$ ผลคือ $(1/10) + (2/10)$

$n=3$ ผลคือ $(1/10) + (2/10) + (3/10)$

จงเขียนโปรแกรมสมบูรณ์ที่มีการเรียกใช้ฟังก์ชันชื่อ SUM ที่มีการส่งผ่านข้อมูล n ใดๆ และส่งผ่านค่ากลับไปยังจุดเรียกใช้



ข้อ 7 จงเขียนโปรแกรมสมบูรณที่มีการเรียกใช้ฟังก์ชันเรียกตัวเองเพื่อทำงานดังนี้

$$(1/1!) + (1 / 2!) + (1/3!) + \dots + (1/n!)$$



END

Recursive Function vs Recursive Mathematic Function

Factorial

$$n! = n * (n - 1) * (n - 2) * (n - 3) * \dots * 3 * 2 * 1$$

$$0! = 1$$

$$1! = 1$$

$$2! = 2 \times 1 = 2$$

$$3! = 3 \times 2 \times 1 = 6$$

$$4! = 4 \times 3 \times 2 \times 1 = 24$$

Factorial Formula

Factorial formula by **Product**

$$n! = \prod_{k=1}^n k$$

Factorial Formula by **Recurrence Relation**

$$n! = \begin{cases} 1 & \text{if } n = 0, \\ (n - 1)! * n & \text{if } n > 0 \end{cases}$$



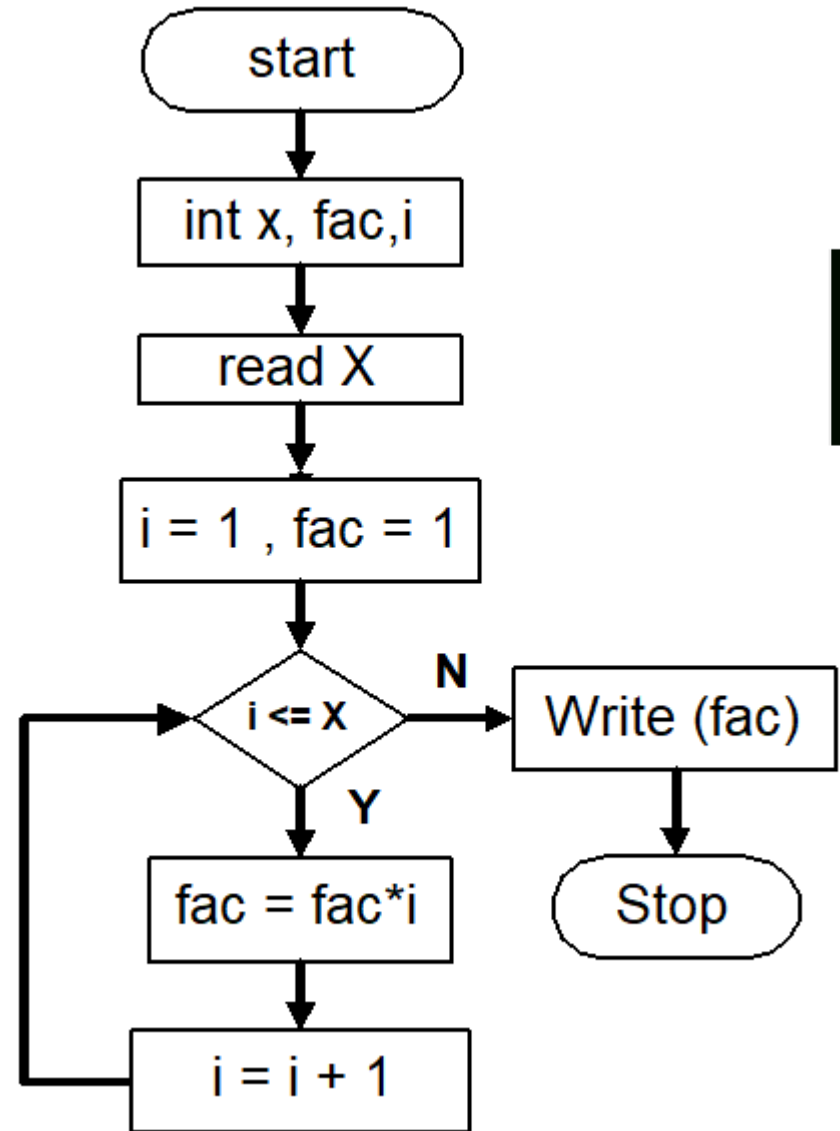
$$n! = \prod_{k=1}^n k$$

main.pas

```

1
2 program FactorialCalculator;
3
4 var
5   x, fac, i: integer;
6
7 begin
8   write('pls, enter data (int): ');
9   readln(x);
10
11   fac := 1;
12   for i := 1 to x do
13   begin
14     fac := fac * i;
15   end;
16
17   writeln('Factorial = ', fac);
18   readln;
19 end.
20

```



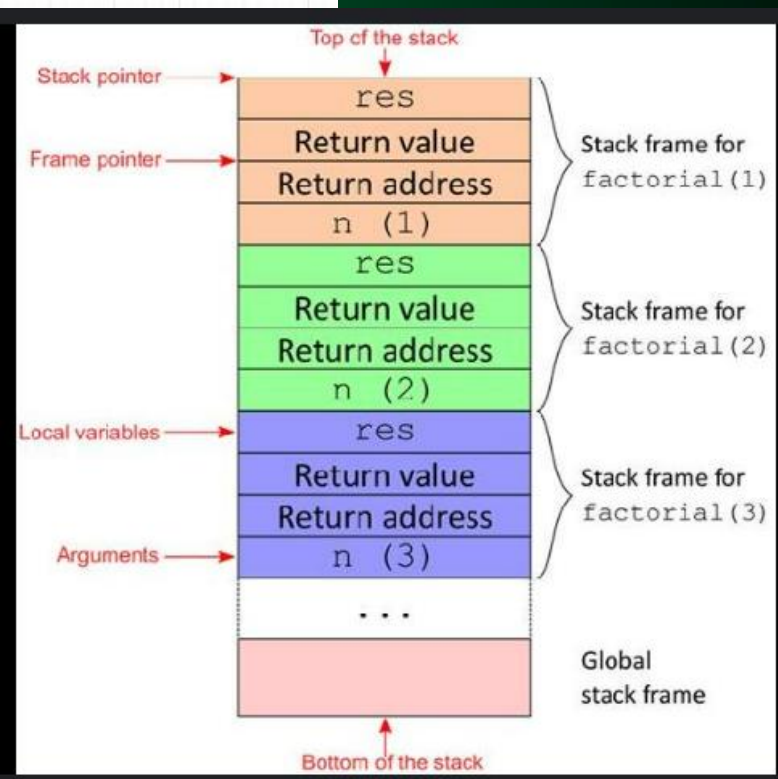
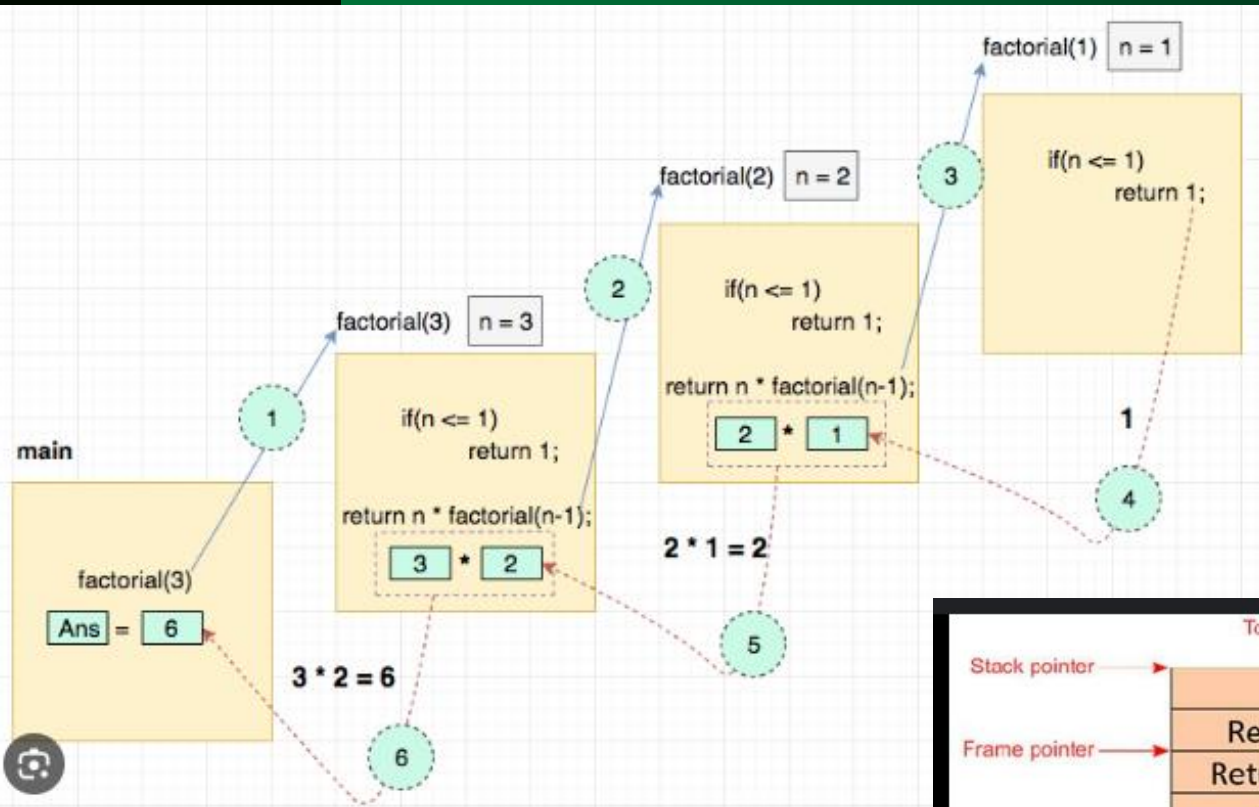
Factorial Formula

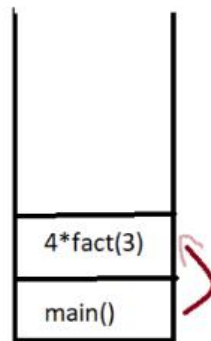
Factorial formula by **Product**

$$n! = \prod_{k=1}^n k$$

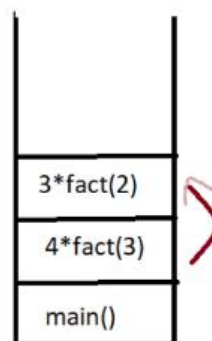
Factorial Formula by **Recurrence Relation**

$$n! = \begin{cases} 1 & \text{if } n = 0, \\ (n-1)! * n & \text{if } n > 0 \end{cases}$$

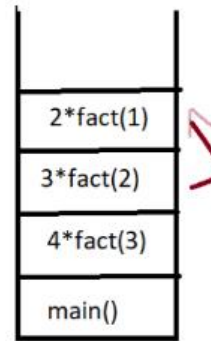




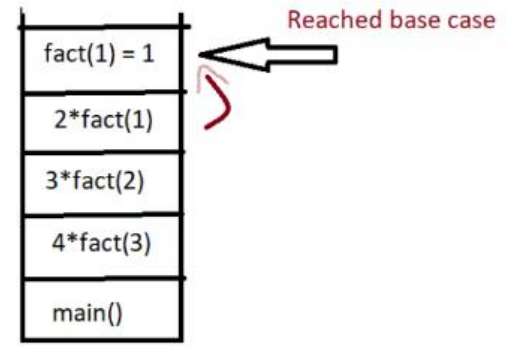
1st call



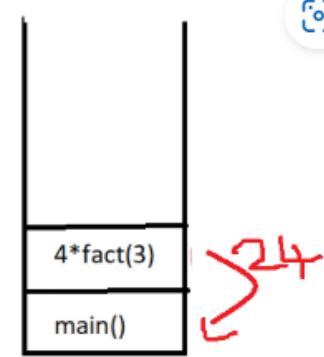
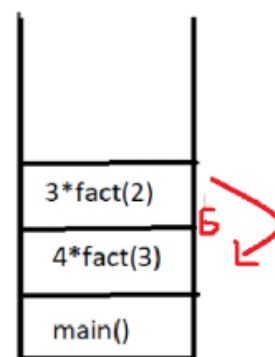
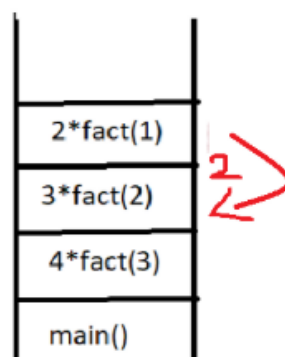
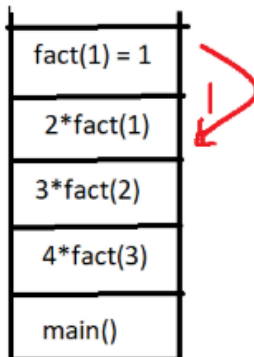
2nd call



3rd call



4th call



มองแบบการทำงานแบบ LOOP โดยกำหนดค่าเริ่มต้นที่ 1

Progressing state until a termination condition

$Pstar(i, n) = \begin{cases} \text{Stop , เมื่อ } i > n \\ \text{write('*'),} \\ \text{Pstar}(i+1, n), \text{ โดย } i \text{ เริ่มที่ } 1 \end{cases}$



Reducing toward a base case

$$\text{Mstar}(n) = \begin{cases} \text{Stop, เมื่อ } n \leq 0 \\ \text{Pstar}(1,n), \\ \text{writeln,} \\ \text{Mstar}(n-1) \end{cases}$$



```
19 lines comp
Line No.1
Line No.2
Line No.3
```

$cv2b(n) = \begin{cases} \text{Stop , เมื่อ } n == 0 \\ \text{Write } (n \% 2) \text{ ev2b}(n/2) \end{cases}$



```
19 lines comp
Line No.1
Line No.2
Line No.3
```

$\text{maxv}(n, \text{ary})$ { $\text{max} = \text{ary}[1]$ เมื่อ $n == 1$
if $\text{arr}[n] < \text{maxv}(n-1, \text{ary})$
 $\text{max} = \text{ary}[n-1];$
Else
 $\text{max} = \text{arr}[n]$



$\text{dif}(n, \text{ary}, \text{arydif}, \text{max}) =$

$\left\{ \begin{array}{l} \text{stop เมื่อ } n \leq 0 \\ \text{dif}(n-1, \text{ary}, \text{arydif}, \text{max}) \\ \text{arrdif}[n] = \text{max} - \text{ary}[n]; \\ \text{Write}(\text{arrdif}[n]); \end{array} \right.$

$\text{arrdif}[n] = \text{max} - \text{ary}[n];$

$\text{dif}(n-1, \text{ary}, \text{arydif}, \text{max})$

$\text{Write}(\text{arrdif}[n]);$

Apirak :